

Iniciado em	sábado, 19 jul 2025, 20:11
Estado	Finalizada
Concluída em	sábado, 19 jul 2025, 20:23
Tempo empregado	12 minutos 7 segundos
Avaliar	10,00 de um máximo de 10,00(100%)

Questão 1

Correto

Atingiu 2,00 de 2,00

Considere que alguém deseja criar uma aplicação que cumprimenta um usuário pelo nome. Abaixo estão indicados os blocos para front-end e back-end, no entanto, falta o conteúdo.

```
ui <- fluidPage(  
  # Conteúdo para criar o front-end.  
)  
server <- function(input, output, session) {  
  # Conteúdo para criar o back-end.  
}  
shinyApp(ui, server)
```

Dados os seguinte blocos enumerados, marque os que devem ser alocadas no bloco front-end (UI) da aplicação.

```
# Bloco 1.  
tableOutput("mortgage")  
  
# Bloco 2.  
output$cumprimento <- renderText({  
  paste0("Olá ", input$nome)  
})  
  
# Bloco 3.  
numericInput("idade", "Qual a sua idade?", value = NA)  
  
# Bloco 4.  
textInput("nome", "Qual o seu nome?")  
  
# Bloco 5.  
textOutput("cumprimento")
```

Escolha uma ou mais:

- ☒ a. Bloco 3 ✖
- ☒ b. Bloco 4 ✔
- ☐ c. Bloco 2
- ☒ d. Bloco 1 ✖
- ☒ e. Bloco 5 ✔

Sua resposta está correta.

As respostas corretas são: Bloco 4, Bloco 5

Questão 2

Correto

Atingiu 2,00 de 2,00

Suponha que você queira criar uma aplicação que permite ao usuário escolher um número (x) entre 1 e 50, e mostra o resultado disso multiplicado por 5. Rode essa aplicação, e perceba que há um erro sobre o objeto x não ser encontrado. O que podemos modificar no local indicado para que a aplicação funcione?

```
library(shiny)

ui <- fluidPage(
  sliderInput("x", label = "Se x é", min = 1, max = 50, value = 30),
  "Então x vezes 5 é",
  textOutput("produto")
)

server <- function(input, output, session) {
  output$produto <- renderText({
    x * 5 # Há algo errado aqui, escolha a alternativa que resolva o problema
  })
}

shinyApp(ui, server)
```

Escolha uma opção:

- ☐ a.

input@x * 5
- ☐ b.

session\$x * 5
- ☐ c.

input[x] * 5
- ☐ d.

`x` * 5
- ☒ e.

input\$x * 5

✔ Essa é a resposta correta.

x

 é uma variável dentro da lista

input

 .

Sua resposta está correta.

A resposta correta é:

input\$x * 5

Questão 3

Correto

Atingiu 2,00 de 2,00

Extenda a aplicação do exercício anterior para permitir que o usuário determine o valor do outro número inteiro a ser multiplicado. O valor de deve ser selecionado da mesma forma que .

Marque as alternativas a seguir sobre o que deve ser modificado no código que forem verdadeiras.

Escolha uma ou mais:

- ☐ a. No bloco UI, deve ser adicionado

```
sliderInput("x", label = "Se y é", min = 1, max = 50, value = 30)
```

- ☐ b. No bloco server, deve ser adicionado

```
sliderInput("y", label = "Se y é", min = 1, max = 50, value = 30)
```

- ☐ c. No bloco server, deve ser adicionado

```
textInput("y", label = "Se y é")
```

- ☐ d. No bloco UI, deve ser adicionado

```
numericInput("x", label = "Se y é", value = 30)
```

- ☐ e. No bloco UI, deve ser adicionado

```
textOutput(exp( y ) * x)
```

- ☒ f. No bloco server, deve ser criado um segundo bloco reativo para receber o input de . ✖

- ☒ g. No bloco UI, deve ser adicionado

```
sliderInput("y", label = "Se y é", min = 1, max = 50, value = 30)
```



- ☒ h. No bloco server, deve-se utilizar os valores de através de dentro do bloco reativo. ✔

Sua resposta está correta.

As respostas corretas são: No bloco server, deve-se utilizar os valores de através de dentro do bloco reativo., No bloco UI, deve ser adicionado

```
sliderInput("y", label = "Se y é", min = 1, max = 50, value = 30)
```

Questão 4

Correto

Atingiu 2,00 de 2,00

Considere o código a seguir, que adiciona mais funcionalidade na aplicação anterior. O que há de novo? Como poderíamos reduzir a quantidade de código duplicado na aplicação utilizando uma expressão reativa?

Abaixo está o código atual.

```
library(shiny)
```

```
ui <- fluidPage(
  sliderInput("x", "Se x é", min = 1, max = 50, value = 30),
  sliderInput("y", "e y é", min = 1, max = 50, value = 5),
  "então, (x * y) é", textOutput("produto"),
  "e, (x * y) + 5 é", textOutput("produto_mais5"),
  "e (x * y) + 10 é", textOutput("produto_mais10")
)
```

```
server <- function(input, output, session) {
  output$produto <- renderText({
    produto <- input$x * input$y
    produto
  })
  output$produto_mais5 <- renderText({
    produto <- input$x * input$y
    produto + 5
  })
  output$produto_mais10 <- renderText({
    produto <- input$x * input$y
    produto + 10
  })
}
```

shinyApp(ui, server)

Marque a alternativa que faz o uso correto de expressões reativas para reduzir código duplicado.

Escolha uma opção:

- ☐ a.
- ```
server <- function(input, output, session) {
 produto <- function() {
 input$x * input$y
 }
 output$produto <- renderText({
 produto()
 })
 output$produto_mais5 <- renderText({
 produto() + 5
 })
 output$produto_mais10 <- renderText({
 produto() + 10
 })
}
```

☐ b.

```
server <- function(input, output, session) {
 produto <- reactive({
 input$x * input$y
 })
 output$produto <- renderText({
 produto
 })
 output$produto_mais5 <- renderText({
 produto + 5
 })
 output$produto_mais10 <- renderText({
 produto + 10
 })
}
```

☒ c.

```
server <- function(input, output, session) {
 produto <- reactive({
 input$x * input$y
 })
 output$produto <- renderText({
 produto()
 })
 output$produto_mais5 <- renderText({
 produto() + 5
 })
 output$produto_mais10 <- renderText({
 produto() + 10
 })
}
```



☐ d.

```
server <- function(input, output, session) {
 produto <- reactive({
 x * y
 })
 output$produto <- renderText({
 produto()
 })
 output$produto_mais5 <- renderText({
 produto() + 5
 })
 output$produto_mais10 <- renderText({
 produto() + 10
 })
}
```

☐ e.

```
server <- function(input, output, session) {
 produto <- reactive({
 list(x = input$x, y = input$y)
 })
 output$produto <- renderText({
 produto()$x * produto()$y
 })
 output$produto_mais5 <- renderText({
 produto()$x * produto()$y + 5
 })
 output$produto_mais10 <- renderText({
 produto()$x * produto()$y + 10
 })
}
```

Sua resposta está correta.

A resposta correta é:

```
server <- function(input, output, session) {
 produto <- reactive({
 input$x * input$y
 })
 output$produto <- renderText({
 produto()
 })
 output$produto_mais5 <- renderText({
 produto() + 5
 })
 output$produto_mais10 <- renderText({
 produto() + 10
 })
}
```



Questão 5

Correto

Atingiu 2,00 de 2,00

A aplicação a seguir é muito similar a uma que você viu anteriormente: um conjunto de dados é selecionado de um pacote (neste caso do pacote ggplot2), e a aplicação retorna um resumo dos dados e um gráfico. O problema é que existem 3 bugs (erros) no código. Marque a(s) alternativa (s) verdadeira(s) sobre os erros.

```
library(shiny)
library(ggplot2)

datasets <- c("economics", "faithfuld", "seals")

ui <- fluidPage(
 selectInput("dataset", "Dataset", choices = datasets),
 verbatimTextOutput("summary"),
 tableOutput("plot")
)

server <- function(input, output, session) {
 # Bloco 1.
 dataset <- reactive({
 get(input$dataset, "package:ggplot2")
 })
 # Bloco 2.
 output$summry <- renderPrint({
 summary(dataset())
 })
 # Bloco 3.
 output$plot <- renderPlot({
 plot(dataset)
 }, res = 96)
}

shinyApp(ui, server)
```

Escolha uma ou mais:

- ☒ a. Existe um erro de digitação no bloco 2, que precisa ser corrigido para `output\$summary`. ✓
- ☐ b. No bloco 2 reativo em server, `renderPrint()` deve ser substituído por `renderTable()`.
- ☒ c. No bloco de UI, `tableOutput()` precisa ser substituído por `plotOutput()`. ✓
- ☒ d. A chamada do dataset no terceiro bloco reativo está errada. O correto seria `dataset()`. ✓
- ☐ e. O vetor `datasets` deveria conter apenas "economics" e "seals".
- ☐ f. Deve ser utilizado `tableOutput()` ao invés de `verbatimTextOutput()`.
- ☐ g. A resolução do gráfico está muito elevada, o argumento `res` precisa ser igual a 72.

Sua resposta está correta.

As respostas corretas são: A chamada do dataset no terceiro bloco reativo está errada. O correto seria `dataset()`, Existe um erro de digitação no bloco 2, que precisa ser corrigido para `output\$summary`, No bloco de UI, `tableOutput()` precisa ser substituído por `plotOutput()`.

